

Project Development Phase

Date: 29 June 2026

Student ID: 24A55A0512

Project Name: Automated Network Request Management

Platform / Domain: ServiceNow / IT Service Management (ITSM) / Network Operations

1. Introduction

The **Automated Network Request Management System** was successfully developed on a ServiceNow personal developer instance (dev280590). The primary objective of this implementation phase was to engineer a robust, automated lifecycle for infrastructure requests (such as VPN provisioning, firewall modifications, and network access changes) by completely removing error-prone email loops and manual hand-offs. The solution delivers cross-functional tracking via the consumer-grade **Service Portal**, secure underlying **Access Control Lists (ACLs)**, and automated operational orchestration powered by **Flow Designer**.

2. Milestone 1 – Application Setup & Portal Initialization

- **Objective:** Establish the development environment framework and front-end service catalog structure.
- **Activities Performed:**
 - Initialized the system architecture to support standard network operations items natively.
 - Designed the front-end catalog taxonomy under the **Service Catalog** → **Network Requests** menu hierarchy.
 - Configured user portal accessibility criteria to display items cleanly via standard widgets.

3. Milestone 2 – Database & Schema Configuration

- **Objective:** Create and index a custom structural logging repository to track historical data transactions alongside baseline ITSM transitional engines.
- **Activities Performed:**
 - Constructed the **Network Database** storage structure labeled `u_network_database`.
 - Provisioned critical data tracking columns to house operational variables cleanly:
 - `u_request_number` (String, tracking baseline REQ numbers)

- u_requested_for (String, indexing the target end-user identity)
- u_device_type (String, mapping hardware scopes)
- u_work_status (String, mapping state sequences)
- u_updates (Integer, tracking transaction iteration layers)
- Set default display mappings and field lengths to preserve structural normalization parameters.

System Configuration Capture – Table Dictionary Structure

The engineered relational table layout below details the active data schema tracking operational records:

![Figure 1: ServiceNow Custom Table Definition Structure]

4. Milestone 3 – User, Role-Based Access Control & Security Matrix

- **Objective:** Define strict security governance profiles via Access Control Lists (ACLs) to regulate read, write, create, and delete actions.
- **Activities Performed:**
 - Provisioned test user records within the sys_user identity engine (e.g., Abel Tuter, Fred Luddy).
 - Generated and configured specific explicit operational rules across all transactional vectors (create, read, write, delete).
 - Structured access constraints ensuring requesters are exclusively granted visibility into items they own, while fulfillment engineers (sn_network_engineer) maintain administrative writing rights over assigned tasks.

System Security Capture – Table Access Control Lists (ACLs)

The security policy configuration below validates that four active rule constraints securely lock down operations against the target table:

![Figure 2: ServiceNow Access Control List Security Configurations]

5. Milestone 4 – User Interface & Dynamic Form Engineering

- **Objective:** Design dynamic Service Portal request forms using advanced field-level controls and interactive validation checks.
- **Activities Performed:**
 - Engineered individual functional catalog forms, including the core **Network Request** management interface.

- Implemented dynamic mandatory parameters (* Indicates required) to block incomplete inputs.
- Formulated field layouts to capture core operational details: "Opened on behalf of", "Email ID", "User Name", "Phone Number", and a "Proof of Document" file attachment upload widget.

System UI Capture – Dynamic Service Portal Intake View

The user-facing intake form features contextual field visibility rules and tooltips that guide users during data entry:

![Figure 3: ServiceNow Service Portal Intake Catalog Item View]

System UI Capture – Enforced Validation Checks & Form Mandatory Logic

This view details client-side validations blocking form submissions until all required network constraints are met:

![Figure 4: ServiceNow Service Portal Mandatory Field Rule Validations]

6. Milestone 5 – Workflow Orchestration & Notification Engine

- **Objective:** Construct asynchronous automated flow rules in Flow Designer to handle sequential multi-level manager reviews, group routing, and SLA execution.
- **Activities Performed:**
 - Programmed a multi-stage execution layout inside **Flow Designer** triggered upon successful catalog checkout.
 - Formulated linear progression logic that automatically updates request states to **Approved** once managerial verification parameters pass.
 - Bound the engine to the system **Notification Engine** to trigger immediate outbound SMTP records tracking structural changes (Request was created, Request was approved).
 - Automated tactical work-order distribution by spinning up catalog tasks (SCTASK) mapped directly to respective technology fulfillment groups (e.g., the *Software assignment group*).

Automation Capture – Outbound Transaction Email Logging

The system log trace below verifies successful execution of background SMTP email dispatches for creations, approvals, and task assignments:

![Figure 5: ServiceNow System Transactional Email Log Monitor]

7. Milestone 6 – Application Testing

- **Objective:** Validate end-to-end operational functionality through comprehensive path testing simulations.

Test Scenario 1: End-User Request Intake Lifecycle

- **Test Activity:** End-user accesses the Service Portal, provides validation data, and places an order for a new provisioning task item.
- **Expected Result:** Ticket records commit smoothly, a tracking page details the generation of an active parent ID number, and initial task workflows execute properly.
- **Status: Passed**

Verification Capture – Live Request Summary Generation

The tracking layout below confirms successful parent ticket allocation (REQ0010003) alongside stage assignment mappings (Assess or Scope Task):

! [Figure 6: ServiceNow Portal Checkout Tracking Confirmation Interface]

Test Scenario 2: Flow Designer State Routing & Approval Processing

- **Test Activity:** System checks incoming request variables and loops back approvals to authorizing entities before handing records off to tech queues.
- **Expected Result:** Request states dynamically update to **Approved** upon approval actions, updating core tracking logs.
- **Status: Passed**

Verification Capture – Request Logging State Ledger

The core record tracker verifies that incoming items (REQ0010003, REQ0010002, REQ0010001) evaluate cleanly to an active **Approved** state sequence:

Test Scenario 3: Automated Fulfillment Task Matrix Generation

- **Test Activity:** Request transitions to an authorized state, firing downstream catalog task generation engines.
- **Expected Result:** A child task entry (SCTASK0010006) is generated automatically and routed straight to the designated engineering assignment group queue.
- **Status: Passed** (*Verified via systemic notification traces to the Software group; see Section 6 logs*).

Test Scenario 4: Role-Based Access Isolation (Negative Testing)

- **Test Activity:** Implements system impersonation changes to verify data-layer access constraints against non-authorized user accounts.

- **Expected Result:** The platform rejects unauthorized read or write access attempts, maintaining secure compartmentalization across all records.
- **Status: Passed**

8. Development Summary

The **Automated Network Request Management System** has successfully moved from the concept phase to a fully working custom application prototype on the ServiceNow platform. By combining dynamic catalog variables with secure data structures and Flow Designer workflows, the application delivers end-to-end automation for infrastructure changes.

All development stages, user stories, security controls, and transaction notifications have been verified through thorough platform testing. This provides a clear, reliable, and production-ready solution for enterprise network operations.